

DSRAC SIS: WMMA

4. assignment

Luka Šveigl, ls6727@student.uni-lj.si, 63200301

May 19, 2024

1 Introduction

As part of the fourth assignment of our Computer systems course, we were required to implement the Blocked general matrix multiplication algorithm (GEMM) using the WMMA API and evaluate it on various different matrix and block/grid sizes. Specifically, we opted to use the WMMA C/C++ API.

2 Methodology

To implement the blocked General Matrix Multiplication (GEMM) algorithm using the WMMA C/C++ API, we first divided the input matrices into smaller blocks to enhance memory access and computational efficiency. Leveraging the WMMA API provided by CUDA, we iterated over these blocks, performing matrix multiplication operations efficiently by storing input matrices and intermediate results in WMMA fragments. This approach maximized GPU parallelism and computational throughput, resulting in significant performance gains compared to traditional methods.

3 Results

N	blockDim.x	blockDim.y	TC	no TC	Speedup
256	32	1	0.017568	0.147520	8.39708561
512	32	1	0.100640	1.598432	15.8826709
1024	32	1	0.867392	11.636512	13.4155168
2048	32	1	5.641440	92.476318	16.3923250
4096	32	1	45.607841	715.492615	15.6879299

Table 1: Results of our experiments of the blocked and naive implementations.

The performance evaluation of the blocked general matrix multiplication algorithm implemented using the WMMA C/C++ CUDA API was conducted

across various problem sizes (N). To fairly evaluate both the naive and the blocked implementations, the block and grid dimensions were set according to the following equations:

$$\begin{aligned}
blockDim.x &= 32 \\
blockDim.y &= 1 \\
gridDimBlocked.x &= \frac{N + (BLOCKSIZE * blockDim.x - 1)/32}{(BLOCKSIZE * blockDim.x)/32} \\
gridDimBlocked.y &= \frac{N + BLOCKSIZE * blockDim.y - 1}{BLOCKSIZE * blockDim.y} \\
gridDimNaive.x &= \frac{N + blockDim.x - 1}{blockDim.x} \\
gridDimNaive.y &= \frac{N + blockDim.y - 1}{blockDim.y}.
\end{aligned}$$

We measured the execution times of both the blocked implementation (TC) and the naive implementation (noTC). Additionally, the speedup achieved by utilizing tensor cores was calculated using the equation

$$speedup = \frac{T_{noTC}}{T_{TC}}.$$

The results of the evaluations are presented in Table 1.

The results show that the blocked implementation outperforms the naive approach across various problem sizes. For the smallest matrix, utilizing tensor cores yields a speedup of 8, while increasing the matrix size leads to speedups increasing to, and plateauing at around 15, showing a substantial increase in performance of the blocked implementation.

4 Discussion

In conclusion, our implementation of the blocked general matrix multiplication algorithm using the WMMA C/C++ CUDA API has yielded significant performance improvements over the naive implementation. Across various problem sizes, we observed speedups consistently, with blocked implementation providing speedups in the range of 8 to 16. We believe that further optimizations of the algorithm are definitely still possible though. These findings underscore the importance of leveraging hardware acceleration and optimizing parallelization strategies for enhancing the performance of matrix multiplication algorithms on CUDA-enabled GPUs.